CSE 213 — Software Engineering

Unit Testing with JUnit, TDD, and Code Reviews

Lesson notes · Exam approach · TF practice with solutions · Quick notes (IJ Mam's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — unit-testing and code-review questions
Lesson 16 — Unit Testing, JUnit, Fixtures and Test-Driven Development — 5 - Unit Testing pp. 1–24
Lesson 17 — Code Reviews: Workflow, Issues, Best Practices, Metrics — 6 - CodeReviews pp. 1–21
Quick Notes — Unit Testing and Code Review — revision

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lesson 16: '5 - Unit Testing.pdf' (24 pages). Lesson 17: '6 - CodeReviews.pdf' (21 pages). TDD is textbook background (asked in 2023-24 and 2016-17). The JUnit 4 test classes were compiled and run (against small stand-ins for org.junit, sources in _build/java/l4): all 20 tests pass.

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q6(a) JUnit test class for isLeapYear (including invalid input) (15) [L16]
2023-24 (CSE 213)	Q3(a) Test-driven development and its benefits (12) [L16]
2022-23 (CSE 307)	Q2(b) Issues commonly identified in code review — three examples (12) [L17]
2021-22 (CSE 307)	Q8(a) Driver and navigator in pair programming as code review (10) [L17]
2020-21 (CSE 307)	Q6(c) Issues commonly identified during code review (8) [L17]
2019-20 (CSE 307)	Q3(b) Limited budget: hire reviewers or more testers? Justify (15) [L17] · Q7(a) Fixtures in unit testing (3 examples); unit test for grading A > 80, B 70–80, C 60–70 with BVA (20) [L16]
2018-19 (CSE 307)	Q1(c) Defect rate and defect density; why checklists in code review (9) [L17]
2017-18 (CSE 307)	Q5(c) Indirect benefits of code review (9) [L17]
2016-17 (CSE 307)	Q3(b) Checklist for reviewing non-functional requirements in code (15) [L17] · Q7(a) Five steps of TDD with an example (15) [L16]
2014-15 (CSE 307)	Q1(d) Code-review scenario for a quick-sort method: participants, sample queries (10) [L17] · Q2(c) Automated tester for a descending float sort: order + same data set (13) [L16]
2013-14 (CSE 307)	Q3(c) Driver code generating data sets for an ascending float sort + the module (12) [L16]
2012-13 (CSE 307)	Q4(a) Four data faults to check in a C-program inspection (6) [L17]
2010-11 (CSE 307)	Q6(e) Test cases for a Queue class (15) [L16]

EXAM FOCUS JUnit test-class writing is ★★★ (2024-25 isLeapYear 15 marks, 2019-20 fixtures + grading BVA 20 marks); TDD (2023-24, 12 marks); code-review issues (2022-23, 2020-21) and review metrics/checklists (2018-19, 2017-18). Learn the JUnit skeleton (@Before, @Test, expected exceptions, assertEquals(expected, actual)) and the nine review issue types.

Lesson 16 — Unit Testing, JUnit, Fixtures and Test-Driven Development

Pages 1–24 (5 - Unit Testing)

Unit tests are white-box tests written by developers to verify small units of program functionality, written concurrently with the implementation (pp. 1–2).

Property of a good unit test (p. 3)	Meaning
Automatic	runs completely by itself, no human input
Atomic	decides pass/fail by itself, no human interpretation
Single responsibility	tests exactly one feature
Independent	runs in isolation from other tests (even of the same function)
Repeatable	same result on every run

JUnit (pp. 4–17)

```
public class MathTest {                      // pp. 7-11 (Math.add / Math.sub)
    private Math calc;
    @Before public void setUp()    { calc = new Math(); }   // fixture, before EACH test
    @After  public void tearDown() { calc = null; }        // clean-up after each test
    @Test public void testAdd() {
        assertEquals(11, calc.add(5, 6));  assertEquals(0, calc.add(0, 0));
        assertEquals(-10, calc.add(-5, -5)); assertEquals(1, calc.add(-5, 6));
    }
    @Test public void testSub() {
        assertEquals(-1, calc.sub(5, 6));  assertEquals(0, calc.sub(0, 0));
        assertEquals(0, calc.sub(-5, -5)); assertEquals(-11, calc.sub(-5, 6));
    }
}
```

Assertion (p. 12)	Fails if
assertTrue(test) / assertFalse(test)	test is false / true
assertEquals(expected, actual)	values not equal (equals)
assertSame / assertNotSame	not the same object / the same object (==)
assertNull(v) / assertNotNull(v)	v not null / v null
fail()	always — e.g. after a call that should have thrown

Every assertion may take a message first: assertEquals("message", expected, actual). Framework features (p. 13): **fixtures** (fixed baseline state so results are repeatable — @Before, @Test, @After), **test suites** (@RunWith + @Suite bundle test classes), **test runners**, JUnit classes.

TRAP Write the **expected** value first (pp. 19–20); give tests long descriptive names (test_addDays_wrapToNextMonth_2, p. 22); use expected-answer objects (assertEquals(new Date(2050,3,1), d), p. 21); messages should say what is being checked, not repeat the values JUnit already prints (pp. 23–24).

Test-driven development (textbook)

1. Add a test for the next small piece of functionality.
2. Run all tests — the new one fails (red).
3. Write the minimum code to make it pass.

4. Run all tests — all pass (green).
5. Refactor the code (and tests), re-run — still green; repeat.

Benefits: every line is covered by a test (safety net for refactoring); forces small, decoupled, testable design; requirements clarified before coding; fewer defects and less debugging; tests document the behaviour; quick feedback.

Practice — Lesson 16

(a) Actual — 2024-25 Q6(a). (b) Actual — 2019-20 Q7(a). (c) Actual — 2023-24 Q3(a) / 2016-17 Q7(a). (d) Actual — 2014-15 Q2(c) / 2013-14 Q3(c). (e) Actual — 2010-11 Q6(e).

(a) isLeapYear(int year) returns true for leap years, false otherwise, and throws IllegalArgumentException if year ≤ 0. Write a complete JUnit test class covering all logically distinct input classes including invalid input; meaningful names and assertions. (15)

(b) What are fixtures in unit testing? Give 3 examples. Write a unit-test method for the partitions of a grading system A > 80, B 70–80, C 60–70, with boundary value analysis. (20)

(c) Describe TDD and its benefits / five steps of TDD demonstrated on an example. (12 / 15)

(d) Write an automated tester (driver) for a module that sorts floating-point data, checking the order and that output and input are the same data set; generate different sizes. (13 / 12)

(e) Design test cases for a Queue class. (15)

✓ Solution

(a) Input classes: divisible by 4 not by 100 (leap); not divisible by 4 (not leap); divisible by 100 not by 400 (not leap); divisible by 400 (leap); boundary 1 and 4; invalid 0 and negative (exception).

```
// 2024-25 Q6(a) — one test per logically distinct input class
import static org.junit.Assert.*;
import org.junit.Test;

public class LeapYearTest {
    @Test
    public void divisibleBy4NotBy100_isLeap() {
        assertTrue("2024 is divisible by 4 but not by 100", LeapYear.isLeapYear(2024));
    }
    @Test
    public void notDivisibleBy4_isNotLeap() {
        assertFalse("2023 is not divisible by 4", LeapYear.isLeapYear(2023));
    }
    @Test
    public void divisibleBy100NotBy400_isNotLeap() {
        assertFalse("1900 is a century year not divisible by 400", LeapYear.isLeapYear(1900));
    }
    @Test
    public void divisibleBy400_isLeap() {
        assertTrue("2000 is divisible by 400", LeapYear.isLeapYear(2000));
    }
    @Test
    public void smallestValidYear_isNotLeap() {
        assertFalse("year 1 (boundary)", LeapYear.isLeapYear(1));
    }
    @Test
    public void firstLeapYear_isLeap() {
        assertTrue("year 4", LeapYear.isLeapYear(4));
    }
    @Test(expected = IllegalArgumentException.class)
    public void zero_throws() {
        LeapYear.isLeapYear(0);
    }
    @Test(expected = IllegalArgumentException.class)
    public void negativeYear_throws() {
        LeapYear.isLeapYear(-4);
    }
}
```

(b) Fixture = fixed, known state created before each test so results are repeatable. Examples: an object under test created in @Before (new Grader / new Math); a set of test data arrays; a temporary file or in-memory database prepared and removed in @After. Partitions and boundaries: A {81, 90, 100}, B {71, 75, 80}, C {61, 65, 70}, below C {60}, invalid {−1, 101}.

```
public class Grader {
    // A: mark > 80, B: 70 < mark <= 80, C: 60 < mark <= 70, otherwise F; valid marks 0..100
    public static char grade(int mark) {
        if (mark < 0 || mark > 100) throw new IllegalArgumentException("mark out of range");
        if (mark > 80) return 'A';
        if (mark > 70) return 'B';
        if (mark > 60) return 'C';
        return 'F';
    }
}
```

```
// 2019-20 Q7(a) — fixture + equivalence partitions + boundary values
import static org.junit.Assert.*;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class GraderTest {
    private int[] aMarks, bMarks, cMarks;

    @Before                                     // fixture: fresh test data before every test
    public void setUp() {
        aMarks = new int[]{81, 90, 100};        // just above 80, nominal, upper limit
        bMarks = new int[]{71, 75, 80};         // just above 70, nominal, boundary 80
        cMarks = new int[]{61, 65, 70};         // just above 60, nominal, boundary 70
    }
    @After
    public void tearDown() { aMarks = bMarks = cMarks = null; }

    @Test
    public void partitionA_above80() {
        for (int m : aMarks) assertEquals("mark " + m, 'A', Grader.grade(m));
    }
    @Test
    public void partitionB_71to80() {
        for (int m : bMarks) assertEquals("mark " + m, 'B', Grader.grade(m));
    }
    @Test
    public void partitionC_61to70() {
        for (int m : cMarks) assertEquals("mark " + m, 'C', Grader.grade(m));
    }
    @Test
    public void boundary60_isNotC() {
        assertEquals("mark 60", 'F', Grader.grade(60));
    }
    @Test(expected = IllegalArgumentException.class)
    public void invalid_aboveRange() { Grader.grade(101); }
    @Test(expected = IllegalArgumentException.class)
    public void invalid_negative() { Grader.grade(-1); }
}
```

(c) Steps and benefits above. Example (stack of int): (1) test push then size() == 1 → fails (no class). (2) red. (3) write Stack with push and size. (4) green. (5) refactor; next test: pop returns last pushed → red → implement pop → green; next test: pop on empty throws → implement → green; refactor array growth.

(d)

```
// 2014-15 Q2(c) / 2013-14 Q3(c) — automated tester (driver) for a float sort: order + same multiset
of data
import java.util.Arrays;
import java.util.Random;

public class SortTester {
    static void sortDescending(float[] a) {              // module under test
        Arrays.sort(a);
        for (int i = 0, j = a.length - 1; i < j; i++, j--) { float t = a[i]; a[i] = a[j]; a[j] = t; }
    }
    static boolean isDescending(float[] a) {
        for (int i = 1; i < a.length; i++) if (a[i - 1] < a[i]) return false;
        return true;
    }
    static boolean sameData(float[] in, float[] out) {   // 'originality': permutation of the input
        float[] x = in.clone(), y = out.clone();
        Arrays.sort(x); Arrays.sort(y);
        return Arrays.equals(x, y);
    }
    public static void main(String[] args) {
        Random r = new Random(42);
        int[] sizes = {0, 1, 2, 10, 1000};               // driver generates different sizes
        boolean all = true;
        for (int n : sizes) {
            float[] in = new float[n];
            for (int i = 0; i < n; i++) in[i] = r.nextInt(50) - 25 + r.nextFloat();   // duplicates
and negatives
            float[] out = in.clone();
            sortDescending(out);
            boolean ok = isDescending(out) && sameData(in, out);
            all &= ok;
            System.out.println("n=" + n + (ok ? " PASS" : " FAIL"));
        }
        System.out.println(all ? "ALL PASS" : "SOME FAILED");
    }
}
```

(e) Cases: new queue empty; FIFO order; wrap-around of a circular buffer; dequeue from empty throws; enqueue on full throws; invalid capacity.

```
// 2010-11 Q6(e) — test cases for a Queue class
import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;

public class QueueTest {
    private BoundedQueue<Integer> q;
    @Before public void setUp() { q = new BoundedQueue<>(3); }

    @Test public void newQueue_isEmpty() { assertTrue(q.isEmpty()); assertEquals(0, q.size()); }
    @Test public void fifoOrder() {
        q.enqueue(1); q.enqueue(2); q.enqueue(3);
        assertEquals(Integer.valueOf(1), q.dequeue());
        assertEquals(Integer.valueOf(2), q.dequeue());
    }
    @Test public void wrapAround_keepsOrder() {
        q.enqueue(1); q.enqueue(2); q.dequeue(); q.enqueue(3); q.enqueue(4);
        assertEquals(Integer.valueOf(2), q.dequeue());
        assertEquals(Integer.valueOf(3), q.dequeue());
        assertEquals(Integer.valueOf(4), q.dequeue());
    }
    @Test(expected = IllegalStateException.class) public void dequeueEmpty_throws() { q.dequeue(); }
    @Test(expected = IllegalStateException.class) public void enqueueFull_throws() {
        q.enqueue(1); q.enqueue(2); q.enqueue(3); q.enqueue(4);
    }
    @Test(expected = IllegalArgumentException.class) public void zeroCapacity_throws() { new
BoundedQueue<Integer>(0); }
}
```

Lesson 17 — Code Reviews: Workflow, Issues, Best Practices, Metrics

Pages 1–21 (6 - CodeReviews)

Opening examples (pp. 1–4): style inconsistency, a silly redundant check, a buffer overflow (unterminated msgbuf), an XSS hole (unescaped text). Code review = 'the process of analyzing code written by a teammate (i.e. a peer) to judge whether it is of sufficient quality to be integrated into the main project codebase' (p. 5).

```
Author writes code -> review request -> Reviewer accepts -> reviews code -> Accepted?
     ^                                                                      | no      | yes
     +---------------------- author edits code <---------------------------+        +--> approval ->
merge
                                                                                  (or abandon)
Simplified (p. 11): local branch -> push to Gerrit/GitHub -> reviewer notified -> approved -> push to
main
```

History (pp. 7–8): Fagan inspection — planning, overview, preparation, meeting, rework, follow-up; improves quality but is process-heavy, meetings are hard to arrange, cost is high and adoption low. Modern tool-based review: Gerrit, Review Board, Phabricator, Crucible (p. 9). Effort (p. 10): about 6 h/week, 10–15 % of developer time — for 50,000 developers at $50/h: 50,000 × 6 × 50 = $15,000,000 per week.

Issues identified during code review (pp. 12–13)	Example
Coding style	inconsistent naming/indentation
Redundant checks	the same null check twice
Project design violation	UI class calls the database directly, bypassing the service layer
Misunderstood requirements	discount applied before tax instead of after
Critical security defects	buffer overflow, SQL injection, XSS
Malicious code	hidden back door / logic bomb
Inadequate input validation	negative quantity accepted
Unsafe methods	strcpy, gets, eval
Lack of exception handling	file read without try/catch or resource close

Best practices (SmartBear, pp. 14–20)

1. Review fewer than 200–400 lines at a time (defect discovery drops after that).
2. Authors annotate the source before the review (where to look first, why each change).
3. Set quantifiable goals and capture metrics: **inspection rate** (LOC reviewed per hour), **defect rate** (defects found per hour of review), **defect density** (defects per LOC, e.g. per KLOC).
4. Use checklists — developers repeat the same mistakes, and omissions are the hardest defects to find.
5. Verify that defects are actually fixed (log, discuss, approve the fix).
6. Foster a positive culture: defects are chances to improve, not a measure of the person.

Other (indirect) benefits (p. 21): project awareness, peer impression formation, consistent coding style, design discussion, knowledge dissemination.

Practice — Lesson 17

(a) Actual — 2022-23 Q2(b) / 2020-21 Q6(c). (b) Actual — 2018-19 Q1(c). (c) Actual — 2017-18 Q5(c). (d) Actual — 2019-20 Q3(b). (e) Actual — 2021-22 Q8(a). (f) Actual — 2016-17 Q3(b). (g) Actual — 2012-13 Q4(a). (h) Actual — 2014-15 Q1(d).

(a) Issues commonly identified during code review — three examples. (12 / 8)

(b) Defect rate and defect density; why checklists are used in code review. (9)

(c) Indirect benefits of code review in a company, with examples. (9)

(d) Limited budget: employ reviewers for SRS, design and code, or hire more (cheaper) testers? Justify. (15)

(e) Driver and navigator in pair programming as a form of code review. (10)

(f) Checklist for a code reviewer to review the non-functional requirements. (15)

(g) Four data faults to check when inspecting a C project. (6)

(h) Scenario of reviewing a quick-sort method: participants and sample queries. (10)

✓ Solution

(a) Any three rows of the table, each with a two-line code example, e.g. unsafe method: strcpy(buf, input) into char buf[8] — buffer overflow → use strncpy/snprintf; inadequate validation: withdraw(amount) without checking amount > 0; lack of exception handling: FileReader opened without try-with-resources.

(b) Defect rate = defects found per hour of review (e.g. 12 defects in 3 h = 4/h). Defect density = defects per unit size (e.g. 12 defects in 2,400 LOC = 5 per KLOC). Checklists: people make the same mistakes repeatedly, and omissions (missing null checks, missing error handling) are hard to notice because the code is not there; a checklist makes reviewers look for them every time and gives consistent reviews.

(c) p. 21 list with examples: project awareness (reviewers learn what changed elsewhere); peer impression formation (trust in good engineers); consistent coding style; design discussion (better solutions proposed); knowledge dissemination (juniors learn, bus factor reduced).

(d) Keep (some) reviewers. Defects found in requirements and design reviews cost far less to fix than defects found by testers after coding (cost grows ~10× per phase); reviews also find defects testing cannot (design violations, maintainability, security holes, missing requirements) and spread knowledge. Testers still needed, so: lightweight peer reviews by the developers themselves (tool-based, checklist, < 400 LOC) instead of dedicated reviewers, reviews focused on SRS and critical modules, and the remaining budget on testers.

(e) The driver types the code; the navigator continuously reviews it — spotting bugs, thinking about design, edge cases and tests — and roles switch often. It is real-time code review: defects are caught as they are written, no separate review meeting is needed, and both share knowledge. Limits: both may share the same blind spots, so a later independent review is still useful for critical code.

(f) Performance (no N+1 queries, efficient algorithms, caching, no blocking calls in UI thread); security (input validation, parameterised queries, output encoding, secrets not hard-coded, least privilege); reliability (exception handling, resources closed, retries/timeouts); usability (clear messages, accessibility); maintainability (naming, small methods, no duplication, comments); portability (no OS-specific paths); scalability (stateless services, connection pooling); logging/auditability.

(g) Variables used before initialisation; constants not named (magic numbers); array bounds not checked (off-by-one, upper bound = size − 1); buffer overflow on character strings (missing '\0', strcpy); pointer faults (dangling or NULL pointers, memory not freed).

(h) Participants: author (explains), moderator/lead, reader, reviewers (peer developer, tester), recorder. Sample queries: Is the pivot choice safe for already-sorted input (worst case $O(n^2)$)? Does the partition handle duplicates and the 0- and 1-element arrays? Can recursion depth overflow the stack for large n? Are the indices lo/hi inclusive and is hi = n − 1 passed? Is the sort stable — is that required? Are there unit tests for empty, sorted, reverse-sorted and all-equal arrays?

Quick Notes — Unit Testing and Code Review

Revision points for Lessons 16–17.

L16 · Unit testing

- Automatic, Atomic, Single responsibility, Independent, Repeatable.
- @Before/@After fixture, @Test, @Test(expected=…), assertEquals(expected, actual).
- TDD: red → green → refactor; benefits: coverage, design, fewer defects.

L17 · Code review

- Workflow: request → review → edit → approve → merge. Fagan inspection six steps.
- Nine issue types; < 400 LOC; annotate; metrics: inspection rate, defect rate, defect density; checklists.
- Indirect benefits: awareness, impressions, style, design discussion, knowledge.